

Gözcü — mülakat notları

(kendim için hazırlanmış kısa hatırlatma: proje ne, neden bu seçimler, sayılar ne söylüyor ve zayıf noktalar neler)

1. Otuz saniyede proje

Arama-kurtarma ekipleri dronla geniş alanların fotoğrafını çekiyor. Fotoğraflar 4000×3000 piksel, aranan insan ise ortalama 60×59 piksel — görüntünün on binde üçü. Bir operatörün yüzlerce böyle fotoğrafa tek tek bakması hem yavaş hem de dikkat gerektiren bir iş.

Gözcü bu fotoğrafları tarayıp **insan adayı olabilecek bölgeleri işaretliyor** ve bir operatörün inceleyebileceği bir arayüzde sunuyor. Karar hep insanda kalıyor; sistem sıraya koyuyor, karar vermiyor.

Üç parçadan oluşuyor: tarama işini kuyruğa alan bir Django servisi, işi arka planda yürüten bir Celery işçisi ve operatörün karar verdiği bir React arayüzü.

2. Asıl teknik problem: küçük hedef

Hazır bir nesne dedektörüne 4000×3000 bir görüntü vererseniz, model onu kendi girdi boyutuna (640 piksel) küçültür. Bu küçültmede 60 piksellik bir insan **9 piksele** iner. O boyutta tespit pratikte imkânsız.

Çözüm: karolama (tiling). Görüntüyü 512×512 piksellik parçalara bölüp her parçayı ayrı tarıyorum, sonra sonuçları birleştiriyorum. Böylece insan kendi ölçeğinde kalıyor. Bunun için SAHI kütüphanesini kullandım.

Bedeli: tek görüntü yerine ~80 karo taranıyor, yani iş ~80 kat artıyor. CPU'da bir kare **medyan 19,9 saniye** sürüyor. Bu yüzden tarama HTTP isteğinin içinde yapılamaz — mimarinin geri kalanı büyük ölçüde bu tek gerçeğe dayanıyor.

3. Neden bu teknolojiler

Seçim	Neden
Django + DRF	Kimlik doğrulama, yetkilendirme, admin ve migration hazır geliyor. Coğrafi veri için GeoDjango'nun PostGIS desteği doğrudan kullanılabilir.
PostgreSQL + PostGIS	"Şu iki bulgu birbirine 50 metreden yakın mı" sorusunu veritabanı kendisi cevaplıyor. Elle mesafe hesabı yazmak yerine test edilmiş bir uygulamayı kullanmak daha doğru.
Celery + Redis	Bir tarama dakikalar sürüyor; kullanıcıyı bekletmem. İstek kuyruğa iş atıp hemen 202 dönüyor, arayüz ilerlemeyi soruyor.
ONNX Runtime	Eğitim PyTorch ile yapıldı ama sunucuda PyTorch'un tamamını taşımak gereksiz. ONNX hafif, taşınabilir ve tek dosya.
SAHI	Karolama, örtüşme ve sonuçların birleştirilmesi çözülmüş bir problem; kendim yazsam aynı hataları baştan yapardım.
React + TypeScript	Tespit kutularının görüntü üzerine oturması, eşik kaydırıcısı, yoklama gibi durum yönetimi gerektiren bir arayüz. TypeScript, API sözleşmesindeki değişiklikleri derleme anında yakalıyor.
Docker Compose	Beş servis var (veritabanı, kuyruk, API, işçi, arayüz). Tek komutla ayağa kalkması hem benim için hem değerlendiren için gerekli.

4. En önemli tasarım kararları

4.1. Eşik veriye pişirilmiyor. Tespitler sabit ve düşük bir güven tabanıyla (0,05) kaydediliyor. Arayüzdeki eşik kaydırıcısı yalnızca **gösterimi** süzüyor, modeli yeniden çalıştırmıyor. Sebebi basit: bir tam tarama pahalı; tek koşudan her eşiği sorabilmek istiyorum.

4.2. Operatörün kararı modelin çıktısına dokunmuyor. İnceleme ayrı bir tabloda duruyor. Aynı alana yazsaydım "modelin ne bulduđu" ile "insanın ne düşündüđu" karışır ve eski ölçümleri bir daha üretemezdim.

4.3. Arayüz koordinat uydurmuyor. Veri kümesindeki 1579 görüntünün **hiçbirinde EXIF GPS yok**. Dosya sırasından veya piksel konumundan enlem-boylam türetebilirdim ama bu ölçüm değil uydurma olurdu ve bir arama ekibini yanlış noktaya yönlendirirdi. Konum yoksa "konum bilgisi mevcut değil" yazıyor.

4.4. Model yüklenemezse sessizce sahte dedektöre düşmüyor. Hata yükseliyor ve kare *failed* oluyor. Model bozukken sistemin "çalışıyor" görünmesi, hiç çalışmamasından kötü.

4.5. İşçi ölürse iş kaybolmuyor. Celery mesajı iş **bittikten sonra** onaylıyor (*acks_late*). Bedeli, bir görevin iki kez çalışabilmesi — bu yüzden işleme adımı idempotent: önce o karenin eski tespitlerini siliyor, sonra yazıyor. İkisi birbirine bağlı; biri olmadan diğeri yanlış.

5. Sonuç: ne ölçtüm, ne çıktı

Önce **taban çizgisi** kurdum: hiç eğitim yapmadan, hazır COCO ağırlığıyla karolamalı tarama. Sonra kendi verimle bir model eğittim (*yolo11n*, 512 piksellik karolar üzerinde) ve **aynı test kümesinde, aynı protokolde** karşılaştırdım.

Koşu	conf	Recall	Yanlış pozitif / görüntü
Taban (eğitim yok)	0,30	0,3804	1,81
Model-512 (kendi verim)	0,53	0,6825	1,75

Karşılaştırmayı eşikte değil, yanlış pozitif bütçesinde yaptım. Bu, projenin en önemli metodolojik kararı: aynı eşikte karşılaştırmak iki modeli aynı yerde karşılaştırmayı garanti etmez. Operatöre yansıyan şey eşiğin sayısı değil, **görüntü başına kaç yanlış kutuya bakmak zorunda kaldığı**. O yüzden tabanın yanlış pozitif yükünü (1,81) sabitleyip modelin o bütçeye sığan noktasını aradım.

Sonuç: aynı yanlış alarm yükü altında recall **0,3804'ten 0,6825'e** çıkıyor — 1,79 kat, 157 görüntüde **293 ek hedef**. Üstelik yanlış pozitif de artmıyor (1,75 karşı 1,81).

Bunu şöyle anlatırım: *"Operatör aynı sayıda yanlış kutuya bakıyor ama daha önce hiç işaretlenmemiş 293 hedefi daha görüyor."*

6. Çok muhtemel sorular ve kısa cevaplar

"Recall ve precision ne demek?" Recall: gerçekten orada olan insanların yüzde kaçını buldum. Precision: bulduğum kutuların yüzde kaçının gerçekten insandı. Arama-kurtarmada recall daha kritik — kaçırılan bir insanın bedeli, fazladan bakılan bir kutudan çok daha ağır. Ben precision yerine **görüntü başına yanlış pozitif** kullandım, çünkü operatörün iş yükünü doğrudan o anlatıyor.

"Neden mAP kullanmadın?" Kullandım — eğitim çıktısında var. Ama onu raporun ana sonucu yapmadım, çünkü eğitim mAP'i karolanmış eğitim kümesinde ölçülüyor; benim sonucum ise tam görüntüler üzerinde, karolamalı çıkarımla, operatörün gördüğü şeyin metriğiyle. İkisini yan yana koymak farklı küme ve farklı protokolü karşılaştırmak olurdu.

"Bu sonuç güvenilir mi?" Ölçtüğüm kadarıyla evet, ama önemli bir kısıtı var: test hedeflerinin **939/970'i tek bir kaynaktan** (ZRI) geliyor ve o kaynak eğitim kümesinde de var. Yani model o kaynağa aşına. Bunu **veri sızıntısıyla karıştırmamak** gerekiyor: 1579 görüntünün SHA-256 özetini hesapladım, bölümler arasında ortak dosya **sıfır**. Sızıntı yok, ama kaynak aşinalığı var ve bunu raporda açıkça yazdım.

"Eğitimi nerede yaptın?" Kaggle'ın ücretsiz GPU'sunda. Yerel makinede sadece CPU var; ölçümleri de CPU'da yaptım, çünkü karşılaştırmanın her iki tarafının aynı donanımda olması gerekiyor.

"ONNX'e geçince doğruluk düştü mü?" Ölçtüm: aynı 157 görüntü, aynı protokol, değişen tek şey çıkarım motoru. Üç eşikte de TP, FN, FP, recall ve precision **birebir aynı** çıktı. En büyük skor farkı 2,29e-06. Ölçtüğüm şey **metrik eşitliği**; bit düzeyinde aynılık değil.

"Sistem çökerse ne olur?" Test ettim: tarama sürerken işçi konteynerini `SIGKILL` ile öldürdüm. Öldürülen kare yeniden teslim edildi ve tamamlandı; 4/4 kare son duruma ulaştı, kayıp kare 0, takılı kare 0, kontrolsüz yinelenen tespit 0.

"Güvenlik tarafında ne yaptın?" Erişim **görev üyeliğine** dayanıyor; üye olmayan kullanıcı 403 değil **404** alıyor, böylece kaynağın var olduğu bilgisi bile sızıyor. Görüntülere doğrudan erişim kapalı — `/media/` yayınlanmıyor, tek yol üyeliği doğrulayan bir API ucu. Kritik işlemler değiştirilemez bir denetim kaydına yazılıyor. Giriş ucunda hız sınırı var. Üretim ayarlarında DEBUG kapalı, CORS açık liste, güvenlik başlıkları açık. **Ama** bunlar "güvenli oldu" demek değil; sızma testi yapılmadı.

"En zayıf nokta ne?" Üçünü dürüstçe sayarım:

1. **Gerçek GPS yok.** Harita altyapısı çalışıyor ama gösterecek gerçek koordinat yok; demodaki koordinatlar sentetik ve her katmanda öyle etiketli.
2. **Deney matrisi eksik.** 320 piksellik karolarla eğitilmiş bir model eğitmedim; karo boyutu ile model ölçeği arasındaki etkileşimi ölçemedim.
3. **Oturum belirteci tarayıcı deposunda.** Backend HTTP-only çerez desteklemiyor; bir XSS açığı oturumun çalınması demek. Bunu kapattım diye sunmuyorum, kabul edilmiş bir sınır olarak yazdım.

"Çalışmayan bir şey oldu mu?" Oldu ve raporda duruyor. Son aşamada "yanlış pozitifler insan faaliyeti olan yerlerde mi yoğunlaşıyor" sorusunu ölçmeye çalıştım. Protokolü sonuçları görmeden sabitledim, körlenmiş bir etiketleme yaptım. Beklenen yönde bir sinyal çıktı **ama** planladığım 220 çiftin yalnızca 45'i geçerli oldu; kendi protokolümün ana bulgu eşiği 100 çiftti. Bu yüzden sonucu "bulgu" değil **"ilginç ama kanıtlanmamış"** diye etiketledim ve üründe hiçbir şeyi değiştirmedim. Örneklemin eksik kalmasının sebebi kendi aracımdaki bir kusurdu: her kayıttan sonra bir adayı atlıyordu. Kusuru buldum, düzelttim ve raporda yazdım.

"Bu proje sahada kullanılabilir mi?" Hayır, ve bunu her yerde yazdım. Eğitim ve araştırma prototipi. Ölçülmüş doğruluğu güvenilebilecek bir seviyenin çok altında, sahada doğrulanmadı, sertifikalı değil. Değeri, problemi doğru kurmasında ve her iddiasının ölçülmüş bir kaynağa dayanmasında.

7. Süreç hakkında anlatılacak şeyler

- **Her sayı bir çıktı dosyasından okunuyor.** Raporda tahmin edilmiş, yuvarlanmış veya hatırlanmış tek bir değer yok. Bir sayı ile CSV çelişirse CSV kazanıyor, metin düzeltiliyor.
- **Kanıt gücü etiketleniyor.** Her iddianın yanında BULGU / HİPOTEZ / ÇÜRÜTÜLDÜ / İLGİNÇ AMA KANITLANMAMIŞ etiketi var. Çürütülen hipotezler silinmiyor; ne tahmin edildiği, nasıl ölçüldüğü ve ne çıktığı yazılıyor.
- **Ölçüm yeniden üretilebilir.** Her CSV kendi koşu bilgisini taşıyor: model, karo boyutu, örtüşme, IoU, cihaz, kütüphane sürümleri, tarih ve çalıştırılan komut.
- **Testler.** Backend, arayüz ve analiz script'leri için ayrı test paketleri var ve hepsi her değişiklikte GitHub Actions üzerinde koşuyor.

8. Terimler — bir cümlelik karşılıklar

Terim	Ne demek
Karolama (tiling)	Büyük görüntüyü küçük parçalara bölüp her parçayı ayrı taramak
Recall	Gerçekten orada olanların kaçını buldum
Precision	Bulduklarımın kaç doğruydum
FP / görüntü	Görüntü başına kaç yanlış kutu — operatörün iş yükü
IoU	İki kutunun ne kadar örtüştüğü; 0,30 ve üstünü "aynı hedef" saydım
conf (güven eşiği)	Modelin bir kutuya verdiği puanın alt sınırı
NMS	Aynı hedefi gösteren üst üste kutulardan en iyisini tutup diğerlerini atmak
mAP	Nesne tespitinde standart ortalama başarı metriği
ONNX	Modeli çerçeveden bağımsız tek dosyada taşıyan biçim
Celery	Uzun işleri arka plana atan iş kuyruğu
PostGIS	PostgreSQL'e coğrafi veri ve mesafe hesabı ekleyen eklenti
JWT	İsteklerde kimlik taşıyan imzalı belirteç
Idempotanslık	Aynı işi iki kez yapmanın sonucu değiştirmemesi
Union-Find	Birbirine yakın noktaları gruplara ayıran veri yapısı